

Smart Urban Planning & AI Decision Support Platform

Architecture Analysis & Production Blueprint

PART 1 — Analysis of the Existing Architecture (Task 1)

1.1 What the current document actually describes

The uploaded architecture ("Intelligent Urban Governance — Training-Based Architecture") is a **single-orchestrator, multi-specialist-agent system**. One Orchestrator Agent receives a planner's natural-language goal (e.g. "reduce congestion + AQI in Ward 12"), fans out to three domain agents (Traffic, Pollution, Energy) — each a thin wrapper around a locally-trained model served over FastAPI — then runs the combined outputs through an Arbitration Agent (NSGA-II multi-objective optimizer), a Policy Synthesis Agent, and a Verification Agent before returning a policy card to the dashboard.

User workflow: planner types a goal → Orchestrator parses it → parallel calls to Traffic/Pollution/Energy agents with ward context → Arbitration produces a Pareto front of trade-offs → Policy Synthesis writes a structured, costed, cited policy card → Verification checks feasibility/compliance → result rendered on a React + Leaflet dashboard.

Data flow: entirely **offline/batch**, by design. Datasets (METR-LA, PeMS-BAY, SUMO-simulated Hyderabad traffic, CPCB air quality, ERA5/Kaggle weather, ASHRAE/UCI energy, OSM road & building graphs) are downloaded once, models are trained offline in a `(training/)` pipeline, and only the exported model artifacts are baked into the model-service containers. At inference time nothing calls an external API — each agent hits its own `(/predict)` endpoint on the internal Docker network.

Backend architecture: Docker Compose microservices — `(traffic-model-service)`, `(pollution-model-service)`, `(energy-model-service)` (each FastAPI + loaded `(.pt)`/`(.pkl)` model), `(orchestrator-service)` (FastAPI + LLM function-calling), `(postgres+postgis)`, `(chromadb)` (RAG store), and a React/Leaflet frontend.

AI pipeline: GNN/LSTM for traffic (DCRNN or GAT+GRU, fallback per-segment LSTM), LSTM/Temporal Fusion Transformer for pollution plus a physics-based Gaussian plume dispersion model for spatial interpolation, and XGBoost/LightGBM (or LSTM) for energy. Cross-domain coupling is explicit: Traffic's predicted volumes feed Pollution as an input feature; Energy's backup-generation emissions feed Pollution's source term.

GIS integration: PostGIS for structured geodata, GeoPandas/osmnx for the Hyderabad road graph and building footprints, Leaflet for the frontend map — but there is no described layer system, overlay management, or heatmap/prediction-surface rendering pipeline. GIS today is "a map with a base layer," not a GIS dashboard.

Database architecture: a single PostgreSQL+PostGIS instance stores datasets, policy history, and a model version log; ChromaDB stores RAG embeddings. No time-series-optimized store, cache, search index, or object store is described.

Module communication: synchronous REST calls over the internal Docker network, with the Orchestrator as the sole hub. There is no message bus, no event stream, no async task queue.

1.2 The single most important finding

The Orchestrator's reasoning layer is specified as "Claude/GPT with function-calling."

This directly contradicts the project's core constraint that no AI capability should depend on an external LLM API — the constraint was stated for the *domain* agents (Traffic/Pollution/Energy), but the Orchestrator itself, which sits at the center of every request, is still wired to a live third-party LLM. For a government deployment this also raises data-residency and reliability concerns (a planner's query about ward-level infrastructure leaving the network to a third-party API). This needs to be resolved before anything else in this blueprint, and Section 12 proposes the fix: replace the Orchestrator's

reasoning with a **trained intent classifier + deterministic task planner** (a small fine-tuned transformer or even a rule-augmented classifier over a fixed intent taxonomy), keeping NSGA-II arbitration exactly as-is. This is not just a compliance fix — a fully self-trained orchestrator is a much stronger technical/viva story than "we wired an LLM to some tools."

1.3 Existing modules

Module	Status
Traffic Agent (GNN/LSTM)	Present, well-specified
Pollution Agent (LSTM/TFT + Gaussian plume)	Present, well-specified
Energy Agent (XGBoost/LSTM)	Present, well-specified
Arbitration Agent (NSGA-II)	Present
Policy Synthesis Agent	Present, schema not detailed
Verification Agent	Present, rules not detailed
Orchestrator	Present but LLM-dependent (see 1.2)
RAG store (ChromaDB)	Present, retrieval scope undefined
PostGIS	Present, used only as storage, not as a GIS engine
Offline training pipeline	Present

1.4 Missing modules

- **AuthN/AuthZ** — nothing described: no login, RBAC, MFA, or departmental permissions for a multi-agency government platform.
- **Weather / contextual-intelligence agent** — the brief explicitly wants weather, floods, disasters, events, land use as supporting signals, but no such agent or data pipeline exists yet.
- **GIS dashboard layer system** — heatmaps, overlays, live alerts, prediction surfaces are all named goals with no supporting architecture.
- **Alerting/notification pipeline** — no SMS/email/push/webhook mechanism for anomaly or severe-weather alerts.
- **Real-time/near-real-time ingestion** — the whole system is batch/offline; there's no IoT sensor or camera-feed ingestion path for anything the platform would eventually need to monitor live (traffic cameras, AQI sensors, smart meters).
- **Audit logging** beyond a bare model version log.
- **Citizen-facing services** (complaints, public dashboards).
- **Emergency response workflow** (only implied via "Emergency Vehicle Routing" as a traffic feature, not a cross-domain module).
- **API gateway / rate limiting**.
- **Object storage** for satellite imagery, raw model artifacts, camera frames (no MinIO/S3-compatible store).
- **Observability stack** — no metrics, tracing, or centralized logging.
- **Message queue / event bus** — every call is synchronous REST, which will not scale to city-wide ingestion.
- **CI/CD and MLOps registry** — model versioning is a manual log entry, not a registry with rollback.

1.5 Weak components

- **Orchestrator = single point of failure and single bottleneck.** No described replica strategy, no fallback path if the LLM API (or its replacement) is unavailable.

- **Manual retraining trigger.** "Weekly, or whenever you manually drop in a new batch" is fine for a demo but is not a production retraining pipeline — no automatic drift-triggered retrain, no CI hook.
- **Confidence intervals are informally derived** ("validation error, which is a nice concrete number to report") rather than calibrated — XGBoost in particular does not natively produce calibrated intervals; this needs an explicit quantile-regression or conformal-prediction wrapper, especially before a government body acts on a number.
- **Synthetic-only Hyderabad traffic data.** SUMO-generated data is a reasonable stand-in for a missing sensor network, but the document has no validation loop that checks the SUMO-trained model against any real observation before it's trusted for a live recommendation.
- **Static Gaussian plume dispersion.** Fine as an MVP physics model, but ignores street-canyon effects and building-induced turbulence — acceptable for a first release, worth flagging as a known limitation rather than a hidden one.
- **RAG scope is unstated.** ChromaDB is present but what it indexes (policy documents? regulations? prior decisions?) and how retrieval quality is evaluated is not defined.

1.6 Scalability issues

- Models are loaded from local files at container startup with no dedicated model-serving layer (Triton/TorchServe/BentoML) — no batching, no GPU sharing, no hot-swap without a redeploy.
- One Postgres instance for datasets, policy history, *and* geodata, with no read replicas or partitioning — will not hold up once multiple wards/cities and IoT-scale history land in it.
- NSGA-II arbitration is run live per request; for common ward/intervention combinations this is repeated, avoidable compute that should be cached or precomputed.
- The Gaussian plume surface is recomputed per request rather than cached as a raster.
- No described strategy for scaling from one city to a multi-tenant, multi-corporation deployment (the eventual government-scale target implied by the brief).

1.7 Security issues

None are described in the source document. For a system aimed at municipal corporations and government agencies, the absence of authentication, RBAC, encryption-in-transit/at-rest, network segmentation beyond "internal Docker network," input validation on the Orchestrator's freeform-text entry point (a real prompt-injection surface if an LLM stays in that role), and audit trails is the single largest gap between this document and a deployable government system. Sections 9 and 12 address this directly.

1.8 Performance bottlenecks

- Sequential Arbitration → Policy Synthesis → Verification chain stacks latency after the parallel domain-agent fan-out.
- No caching layer (Redis) for repeated ward/intervention queries.
- RAG retrieval latency and quality are unaddressed.
- Dispersion-surface computation is per-request rather than precomputed/cached.

1.9 Future limitations

- Three domains are hardcoded into the pipeline; adding a fourth (water, waste, health) means re-architecting rather than registering a new agent.
- SUMO-only training data will not reflect real anomalies (accidents, festivals, protests) without a continual real-world grounding mechanism.
- The design implicitly assumes one city/one ward context; there's no multi-tenant partitioning for scaling across multiple municipal corporations, which is the platform's own stated long-term audience.

1.10 AI integration opportunities

- Replace the Orchestrator's LLM dependency with a **trained intent classifier + deterministic planner** — resolves the core contradiction and strengthens the "fully self-trained" pitch (Section 12).
 - Add a **per-domain anomaly-detection layer** (autoencoder or isolation forest on each domain's live feature stream) to power "Automatic Domain Monitoring" (Task 3.2 in the brief).
 - Add a **computer-vision agent** (YOLO-class detector on traffic-camera frames) for accident detection and vehicle/pedestrian density — currently the Traffic domain is pure time-series, with no visual grounding.
 - Add a **Weather Agent** (own trained LSTM/TFT on IMD/ERA5 data) as a fourth first-class agent whose forecast feeds Traffic/Pollution/Energy as an extra input feature — this directly satisfies both Task 3.1 (Weather Intelligence) and the brief's "supporting city factors" requirement, and it keeps the whole system on the same "locally trained models only" footing as the rest of the architecture.
-

PART 2 — New Feature Set 1 (Task 2): 14 Production-Ready Features

Each feature integrates with Traffic, Pollution, and/or Energy rather than standing alone.

2.1 Weather-Linked Multi-Domain Impact Forecaster

- **Problem:** severe weather affects all three domains but nothing today connects a weather forecast to a traffic/AQI/energy consequence.
- **Government benefit:** one advisory instead of three uncoordinated ones; earlier resource pre-positioning.
- **Workflow:** planner opens Weather tab → sees 24h forecast → sees auto-generated "if storm hits, expect X% congestion rise / Y AQI change / Z% demand spike."
- **AI components:** Weather Agent (forecasting) → impact-translation layer (regression heads trained on historical weather-vs-domain-outcome pairs).
- **Datasets:** IMD/ERA5 historical weather, paired historically with CPCB AQI, traffic speed logs, and energy load.
- **Models:** LSTM/TFT for weather; gradient-boosted regressors mapping weather deltas → domain-outcome deltas.
- **Outputs:** 24h weather forecast + per-domain impact estimate with confidence band.
- **Dashboard:** timeline strip + three small impact gauges (traffic/AQI/energy).
- **Complexity:** Medium. **Est. time:** 5–6 weeks.

2.2 Flood Risk & Waterlogging Early Warning

- **Problem:** low-lying wards flood repeatedly with no predictive lead time.
- **Government benefit:** pre-deployment of pumps/barricades, proactive road closure.
- **Workflow:** system flags high-risk drains/wards → officials pre-position crews → closure auto-suggested to Traffic Agent.
- **AI components:** rainfall-runoff regression + drainage-capacity graph model.
- **Datasets:** rainfall history, drain network GIS, elevation (DEM), past waterlogging complaint logs.
- **Models:** gradient-boosted regressor on rainfall+terrain+drainage features; optional GNN over the drainage graph.
- **Outputs:** per-ward flood-risk score, ranked hotspot list.
- **Dashboard:** GIS choropleth with risk shading.
- **Complexity:** Medium-High. **Est. time:** 6–8 weeks.

2.3 Urban Heat Island Monitoring & Cooling Advisory

- **Problem:** hard surfaces and low green cover create localized heat spikes affecting health and energy demand.

- **Government benefit:** target tree-planting/cool-roof programs where they matter most; anticipate cooling-driven load spikes.
- **Workflow:** heat-map surfaces updated seasonally → planners select intervention zones → Energy Agent estimates demand relief.
- **AI components:** land-surface-temperature regression from satellite bands + land-use.
- **Datasets:** Landsat/Sentinel thermal bands, land-use/land-cover, building density.
- **Models:** random forest / CNN on raster tiles.
- **Outputs:** heat-island severity raster, ranked ward list.
- **Dashboard:** GIS heatmap layer.
- **Complexity:** Medium. **Est. time:** 5 weeks.

2.4 Emergency Vehicle Green-Corridor Routing

- **Problem:** ambulances/fire trucks lose time to un-adapted signal timing.
- **Government benefit:** measurable reduction in emergency response time.
- **Workflow:** dispatch triggers request → Traffic Agent computes a live-cleared corridor → signals along the route re-time.
- **AI components:** shortest-time routing over the live congestion graph + signal-priority policy.
- **Datasets:** road graph, live/simulated congestion state, signal timing plans.
- **Models:** reuses Traffic Agent's GNN forecast; corridor selection via constrained shortest path (Dijkstra/A* on predicted travel time).
- **Outputs:** recommended corridor + signal-retiming plan.
- **Dashboard:** live route overlay with ETA.
- **Complexity:** Medium. **Est. time:** 4–5 weeks.

2.5 Public Transport Demand & Delay Prediction

- **Problem:** buses bunch or run empty with no demand foresight.
- **Government benefit:** better fleet allocation, reduced wait times.
- **Workflow:** planner views predicted ridership/delay per route-hour → adjusts frequency.
- **AI components:** ridership time-series model conditioned on traffic forecast.
- **Datasets:** GTFS/ticketing logs, traffic congestion forecast, calendar features.
- **Models:** LSTM/GBM on route-hour ridership + delay.
- **Outputs:** predicted ridership and delay per route.
- **Dashboard:** route timeline heat strip.
- **Complexity:** Medium. **Est. time:** 5 weeks.

2.6 Smart Parking Availability Prediction

- **Problem:** drivers circle for parking, adding to congestion and emissions.
- **Government benefit:** reduced downtown congestion and idling emissions.
- **Workflow:** citizen/planner app shows predicted availability per lot for the next hour.
- **AI components:** occupancy forecasting per lot.
- **Datasets:** historical occupancy/sensor logs, event calendar, nearby traffic volume.
- **Models:** GBM or LSTM per lot cluster.
- **Outputs:** predicted occupancy %, confidence.
- **Dashboard:** color-coded lot markers on GIS map.
- **Complexity:** Low-Medium. **Est. time:** 3–4 weeks.

2.7 Street Light Fault & Energy Optimization

- **Problem:** faults go undetected for days; fixed-schedule lighting wastes energy.
- **Government benefit:** direct energy savings, faster fault resolution.
- **Workflow:** anomalous consumption per pole flags a probable fault → work order auto-generated; dimming schedule optimized by ambient/traffic data.

- **AI components:** per-pole consumption anomaly detector + adaptive dimming policy.
- **Datasets:** smart-meter/pole consumption logs, ambient light sensor data, traffic density.
- **Models:** isolation forest / autoencoder for fault detection; rule-based or RL-lite dimming policy.
- **Outputs:** fault list, dimming schedule.
- **Dashboard:** map markers colored by status (OK/fault/dimmed).
- **Complexity:** Medium. **Est. time:** 5 weeks.

2.8 Industrial Emission Compliance Monitoring

- **Problem:** industrial sources are a major but poorly tracked pollution contributor.
- **Government benefit:** targeted enforcement, defensible evidence trail.
- **Workflow:** facility emission trend flagged as anomalous → inspection recommended → linked to Pollution Agent's source-attribution output.
- **AI components:** per-facility emission trend model + threshold/anomaly classifier.
- **Datasets:** CPCB industrial monitoring feeds (where available), facility registry, wind data for source attribution.
- **Models:** time-series regression + change-point detection.
- **Outputs:** facility risk score, trend chart.
- **Dashboard:** facility list + trend sparkline.
- **Complexity:** Medium. **Est. time:** 5–6 weeks.

2.9 Disaster Preparedness "What-If" Simulation

- **Problem:** officials plan disaster response reactively rather than by simulated scenario.
- **Government benefit:** pre-tested evacuation/resource plans before an actual event.
- **Workflow:** planner selects a scenario (flood/heatwave/earthquake) → system simulates traffic evacuation load, pollution spikes, and grid stress together.
- **AI components:** combines all three trained domain models under a synthetic scenario input, run through Arbitration.
- **Datasets:** same as domain agents, plus historical disaster case studies where available.
- **Models:** reuses Traffic/Pollution/Energy models with scenario-perturbed inputs.
- **Outputs:** simulated multi-domain impact report.
- **Dashboard:** scenario picker + before/after comparison map.
- **Complexity:** High. **Est. time:** 8–10 weeks.

2.10 Citizen Complaint Triage & Routing

- **Problem:** complaints (potholes, streetlight outages, garbage, water logging) arrive unstructured and get manually sorted.
- **Government benefit:** faster routing to the right department, priority scoring.
- **Workflow:** citizen submits complaint with photo/text/location → system classifies category and urgency → routes to department, cross-references live domain state (e.g. a "flooding" complaint in an already-flagged high-risk drain gets boosted priority).
- **AI components:** trained text classifier (not an LLM API — a fine-tuned local transformer or classical NLP classifier) + image classifier for photo evidence.
- **Datasets:** historical complaint logs with department/resolution labels.
- **Models:** fine-tuned local DistilBERT-class classifier; CNN for photo category.
- **Outputs:** category, urgency score, routed department.
- **Dashboard:** complaint queue with map pins.
- **Complexity:** Medium. **Est. time:** 6 weeks.

2.11 EV Charging Infrastructure Load Balancer

- **Problem:** uncoordinated EV charging worsens peak demand.
- **Government benefit:** supports EV adoption without destabilizing the grid.

- **Workflow:** charging stations report load → Energy Agent recommends staggered charging windows → integrated with peak-load forecast.
- **AI components:** charging-demand forecast + scheduling optimizer.
- **Datasets:** station usage logs, grid load forecast (from Energy Agent), EV adoption trend data.
- **Models:** GBM demand forecast + linear/MILP scheduling optimizer.
- **Outputs:** recommended charging windows per station.
- **Dashboard:** station status + load timeline.
- **Complexity:** Medium. **Est. time:** 5–6 weeks.

2.12 Construction Impact & Road Closure Advisory

- **Problem:** construction projects are approved without systematic traffic/pollution impact review.
- **Government benefit:** informed approvals, pre-planned detours.
- **Workflow:** planner submits a proposed closure → Traffic Agent simulates rerouted congestion, Pollution Agent estimates dust/emission impact.
- **AI components:** reuses Traffic + Pollution agents with a closure scenario input.
- **Datasets:** road graph, historical construction-period traffic/AQI data.
- **Models:** scenario-perturbed Traffic/Pollution forecasts.
- **Outputs:** impact report + suggested detour.
- **Dashboard:** before/after congestion map.
- **Complexity:** Medium. **Est. time:** 5 weeks.

2.13 Noise Pollution Mapping & Zoning Advisory

- **Problem:** noise complaints are common but noise levels are rarely mapped systematically.
- **Government benefit:** evidence base for zoning and enforcement.
- **Workflow:** noise surface generated from sensor/traffic-derived estimates → overlaid with residential/school zones → hotspots flagged.
- **AI components:** noise-level regression from traffic volume + land use (where dedicated sensors are absent).
- **Datasets:** available noise sensor logs (if any), traffic volume, land-use map.
- **Models:** regression model trained on any available sensor ground truth, traffic volume as primary proxy feature.
- **Outputs:** noise surface, hotspot list near sensitive zones.
- **Dashboard:** GIS overlay.
- **Complexity:** Low-Medium. **Est. time:** 4 weeks.

2.14 Power Outage Prediction & Restoration Prioritization

- **Problem:** outages are handled reactively; restoration order isn't risk-weighted.
 - **Government benefit:** faster restoration to critical infrastructure (hospitals, water pumping stations).
 - **Workflow:** outage-risk score per feeder computed from load/weather → restoration crews prioritized by a criticality-weighted score after an outage occurs.
 - **AI components:** outage-risk classifier + criticality-weighted restoration ranking.
 - **Datasets:** outage history, feeder load, weather, critical-facility registry.
 - **Models:** gradient-boosted classifier for outage risk; weighted ranking function for restoration order.
 - **Outputs:** risk map, restoration priority list.
 - **Dashboard:** feeder map colored by risk + ranked restoration list.
 - **Complexity:** Medium. **Est. time:** 5–6 weeks.
-

PART 3 — Expansion of Existing Ideas (Task 3)

3.1 Weather Intelligence Module

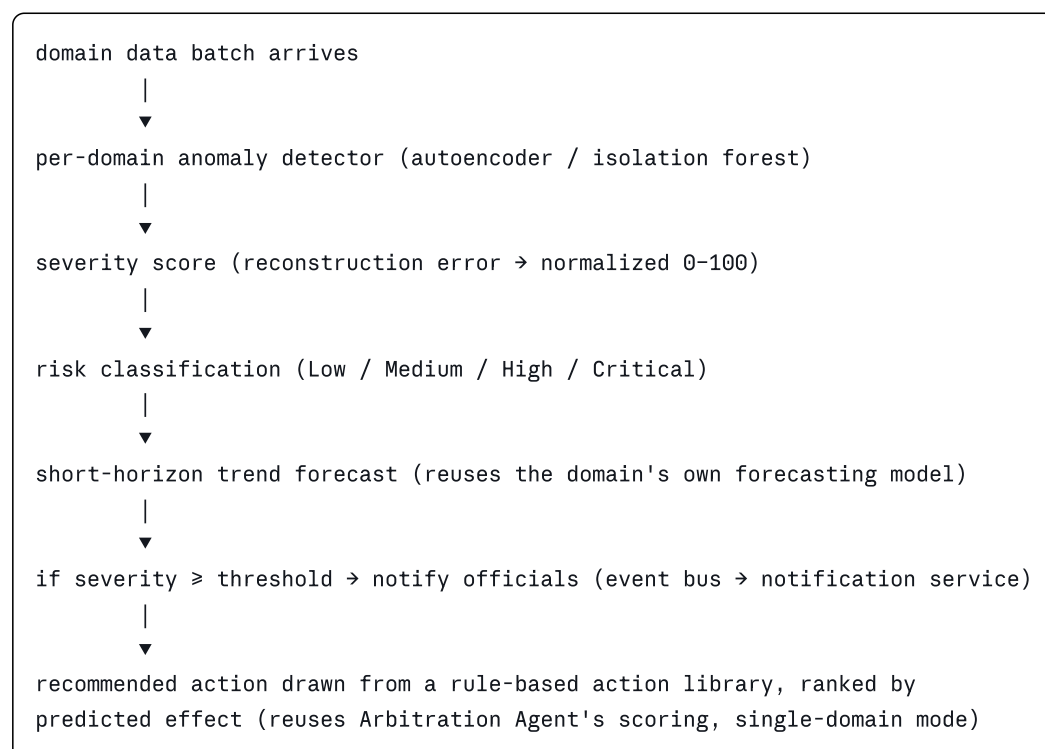
Displays (next 24h): rainfall probability/amount, temperature, humidity, wind speed, storm warnings, flood probability, lightning alerts.

Automatic severe-weather response: when a forecast crosses a severity threshold, the module (a) raises an alert automatically, (b) feeds the forecast into Traffic/Pollution/Energy as an extra input feature so each domain's own forecast already reflects the weather shift, and (c) surfaces a plain-language impact summary generated from the regression layer in Section 2.1 — not from an LLM, from the trained impact-translation model.

- **Datasets:** IMD station data / ERA5 reanalysis (bulk, downloaded once), lightning-strike open datasets where available.
- **Prediction models:** LSTM or Temporal Fusion Transformer for the core weather variables; a separate lightweight classifier for storm/flood/lightning alert triggers (trained on historical threshold-crossing events).
- **Training pipeline:** identical pattern to Traffic/Pollution/Energy — bulk historical download → chronological train/val/test split → offline training → exported artifact → served from its own `weather-model-service` container.
- **Visualization:** 24h timeline strip with per-hour icons; a severity badge that turns amber/red when a threshold is crossed.
- **Alert mechanism:** threshold-crossing triggers a message onto the internal event bus (Section 12 adds this), which both the notification service and the three domain agents subscribe to.

3.2 Automatic Domain Monitoring

Continuous monitoring workflow (Traffic / Pollution / Energy):



Each domain agent is extended with an anomaly-detection head trained on its own historical feature stream (no new dataset needed beyond what each agent already trains on), so this bolts onto the existing Traffic/Pollution/Energy agents rather than requiring a new domain.

3.3 Traffic Intelligence (expanded)

Adds accident detection, congestion prediction, delay estimation, density prediction, emergency routing, signal optimization, road-blockage prediction, and parking prediction on top of the existing GNN/LSTM forecaster. Accident detection specifically benefits from

the computer-vision agent proposed in Section 1.10 (camera-based detection) combined with a sudden-speed-drop signal from the existing time-series model — using both catches accidents that either signal alone would miss.

Rerouting-plan generation: on a detected accident, the Traffic Agent generates several candidate reroutes (via k-shortest-paths over the predicted-travel-time graph), each scored on:

Criterion	Source
Travel time	Traffic Agent forecast
Congestion reduction	Delta vs. do-nothing baseline
Fuel savings	Distance × congestion-adjusted consumption model
Emergency accessibility	Road-class/width check against emergency vehicle requirements
Road capacity	Static road-graph attribute
Environmental impact	Fed to Pollution Agent as a volume-swap scenario
Confidence score	Model validation-error-derived interval
Overall AI score	Weighted sum, tunable by planners
Reasoning	Templated explanation generated from the feature contributions (not free-text LLM output)

Planners compare candidates side-by-side and approve one — this maps directly onto the existing Policy Synthesis Agent's schema.

3.4 Pollution Intelligence (expanded)

AQI prediction, hotspot detection, source identification, emission trend forecasting, industrial monitoring, and noise mapping build on the existing LSTM/TFT + Gaussian plume design. **Mitigation-plan generation** mirrors the traffic scoring table above, scored on: expected AQI improvement, affected population (from population-density raster × affected area), estimated cost, implementation time, confidence score, environmental impact, and a ranking with templated reasoning — again feeding the same Arbitration → Policy Synthesis pipeline already in the architecture, so no new orchestration logic is needed, only new candidate-generation logic per domain.

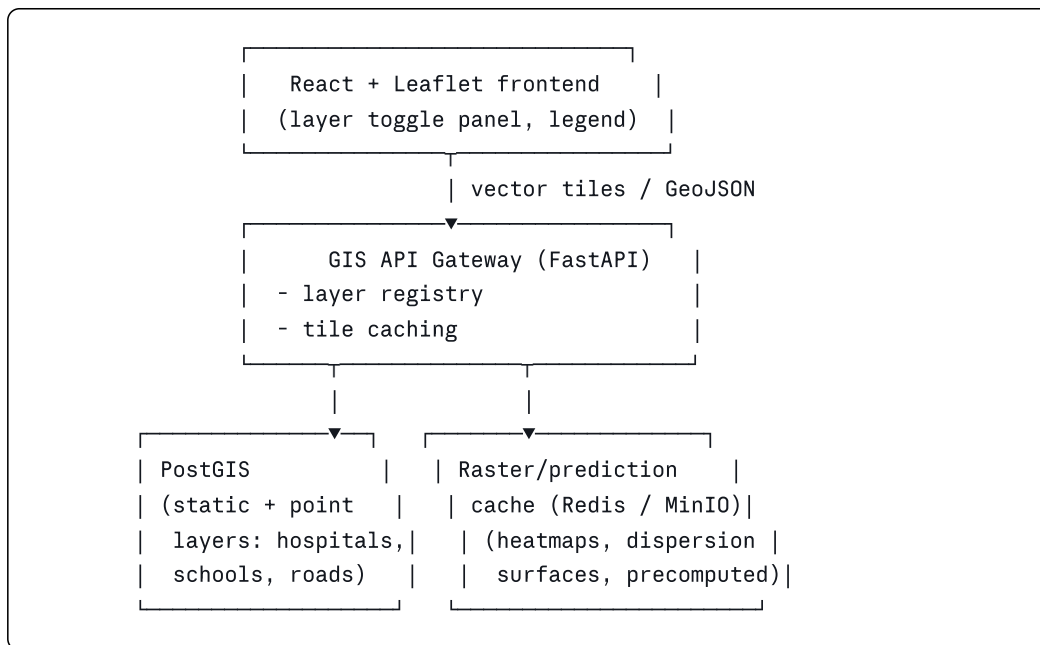
3.5 Energy Intelligence (expanded)

Demand forecasting, peak-load prediction, smart-grid monitoring, renewable optimization, building analysis, outage prediction, and street-light optimization build on the existing XGBoost/LSTM design. **Optimization-plan generation** follows the same pattern: expected energy savings, cost reduction, carbon reduction, implementation time, confidence score, ranking, reasoning — same Arbitration/Policy Synthesis reuse.

3.6 Live GIS Dashboard

Layers: traffic, pollution, energy infrastructure, weather, rainfall, flood zones, hospitals, police, fire stations, schools, power stations, substations, construction zones, road closures, satellite view, heatmaps, live alerts, prediction overlays.

Architecture:



Static/point layers (hospitals, schools, police, fire, substations, closures) are stored and served directly from PostGIS as vector tiles. Model-derived surfaces (AQI dispersion, flood risk, heat islands, congestion heatmaps) are **precomputed on a schedule** into raster tiles cached in Redis/MinIO rather than recomputed per request — this is the fix for the "dispersion recomputed per request" bottleneck flagged in Section 1.8. Live alerts are pushed to the frontend over a WebSocket channel fed by the event bus described in Section 12.

PART 4 — Additional Advanced Features (Task 4): 14 More Features

4.1 Water Logging Detection via Camera + Sensor Fusion

Combines low-cost ultrasonic water-level sensors with CV-based detection from traffic cameras. **Why needed:** rainfall-runoff models (2.2) predict *risk*; this detects *actual* waterlogging in near-real-time. **Datasets:** sensor logs, camera frames. **Models:** threshold classifier + lightweight CNN. **Complexity:** Medium.

4.2 Crowd Density Prediction (Public Events & Festivals)

Predicts crowd buildup at event venues from historical attendance, ticketing, and transit-boarding data, feeding pre-emptive Traffic signal and Public Transport adjustments. **Models:** GBM/LSTM on historical event-attendance patterns. **Complexity:** Medium.

4.3 Festival & Special-Event Traffic Management

Extends 4.2 with a scenario-mode of the Traffic Agent that pre-computes signal and routing plans for known recurring festivals. **Complexity:** Medium.

4.4 Illegal Construction / Land-Use Change Detection

Satellite-imagery change detection flags new structures inconsistent with the zoning layer. **Models:** CNN-based change detection on paired satellite scenes (before/after). **Complexity:** High (needs a labeled change-detection dataset or transfer learning from public benchmarks).

4.5 Infrastructure Health Monitoring (Roads/Bridges)

Uses road-surface condition data (from vehicle vibration proxies or periodic imagery) to predict maintenance need before failure. **Models:** regression on surface roughness index + traffic load. **Complexity:** Medium-High.

4.6 Road Maintenance Prediction

Predicts pothole/degradation likelihood per segment from age, traffic load, and weather exposure, ranking segments for proactive repair budgets. **Models:** survival-analysis-style regression (time-to-failure). **Complexity:** Medium.

4.7 Green Zone / Urban Forestry Recommendation

Recommends tree-planting locations that maximize both heat-island relief (2.3) and pollution absorption, subject to available public land. **Models:** spatial optimization (greedy or MILP) over the heat + pollution rasters. **Complexity:** Medium.

4.8 Public Complaint Intelligence (Aggregate Trends)

Aggregates the per-complaint triage output (2.10) into ward-level trend dashboards so officials see systemic issues, not just individual tickets. **Models:** none new — aggregation + trend detection on existing classifier outputs. **Complexity:** Low.

4.9 Solar Rooftop Potential Estimation

Estimates rooftop solar generation potential per building from footprint area, orientation (from satellite/LiDAR where available), and local irradiance, supporting renewable-energy planning. **Models:** physical irradiance model + regression correction. **Complexity:** Medium.

4.10 Water Supply Demand Forecasting

Adjacent to Energy's load forecasting pattern — predicts municipal water demand per zone from population, weather, and historical consumption, useful for pump scheduling (which itself affects Energy load). **Models:** LSTM/GBM. **Complexity:** Medium.

4.11 Street Vendor / Encroachment Zone Analysis

Flags footpath/road encroachment from periodic street-view imagery, cross-referenced with traffic-flow impact. **Models:** object detection (CV) + rule-based impact scoring. **Complexity:** Medium-High.

4.12 School-Zone Safety Scoring

Combines traffic speed near schools, accident history, and lighting/streetlight status (2.7) into a safety score used to prioritize crossing-guard placement or speed-bump installation. **Models:** weighted scoring function over existing model outputs — no new model required. **Complexity:** Low.

4.13 Multi-Ward Resource Allocation Optimizer

Given a fixed city budget, allocates it across the ranked interventions produced by Traffic/Pollution/Energy mitigation plans (3.3–3.5) to maximize total benefit — an extension of the Arbitration Agent to a portfolio-selection problem across wards rather than one ward at a time. **Models:** reuses NSGA-II / adds a knapsack-style budget constraint. **Complexity:** High.

4.14 Post-Intervention Impact Verification

After an approved policy is implemented, compares actual outcomes (next data batch) against the policy card's predicted impact, closing the loop that Section 1.5 flagged as missing — this is what actually validates the synthetic-trained Traffic model against reality over time. **Models:** none new — a scheduled comparison job against the retraining/drift pipeline in Section 3.7 of the original document. **Complexity:** Medium.

PART 5 — AI Model Recommendations by Module (Task 5)

Module	Model Type	Input Features	Output	Training Strategy	Eval Metrics	Expected Accuracy*
Traffic forecasting	DCRNN (GNN) / GAT+GRU fallback	speed, volume, hour, day, calendar	speed/volume per segment, next T steps	offline, chronological split	MAE, MAPE	MAPE ~8–12% (in line with published METR-LA benchmarks)
Pollution forecasting	LSTM / Temporal Fusion Transformer	pollutants, weather, traffic volume	AQI/pollutant forecast per station	offline, chronological split	RMSE, MAE	RMSE within CPCB station-class tolerance
Pollution dispersion	Gaussian plume (physics-based)	station forecast + wind vector	pollution surface	none (physics model)	visual/field validation	n/a
Energy forecasting	XGBoost / LightGBM (or LSTM)	weather, calendar, building metadata, load history	load forecast, savings estimate	offline, chronological split	MAPE, RMSE	MAPE ~5–10% (typical for ASHRAE-class tabular GBM)
Weather forecasting	LSTM / TFT	station history, calendar	rainfall/temp/wind/storm forecast	offline, chronological split	RMSE, Brier score (for storm probability)	domain-typical for short-horizon nowcasting
Anomaly/domain monitoring	Isolation Forest / Autoencoder	domain feature stream	severity score, risk class	unsupervised, retrained per batch	precision/recall vs. labeled incidents	n/a (unsupervised)
CV accident/vehicle detection	YOLO-class detector	camera frames	bounding boxes, event flags	transfer learning + fine-tune on local footage	mAP	mAP dependent on fine-tuning data volume
Flood risk	Gradient-boosted regressor / GNN over drainage graph	rainfall, terrain, drainage capacity	risk score per ward	offline, chronological	AUC, MAE	task-dependent
Complaint triage (text)	Fine-tuned local transformer (e.g. DistilBERT-class)	complaint text	category, urgency	supervised, fine-tune on labeled complaints	F1, accuracy	80%+ achievable with a few thousand labeled examples
Complaint triage (image)	CNN classifier	complaint photo	category	supervised	F1	task-dependent
Arbitration	NSGA-II (pymoo)	candidate interventions + effect vectors	Pareto front	n/a (optimizer, not learned)	hypervolume	n/a

Module	Model Type	Input Features	Output	Training Strategy	Eval Metrics	Expected Accuracy*
Orchestrator planner (replacement)	Fine-tuned intent classifier + rule-based planner	planner's structured/free-text request	task plan (which agents, what parameters)	supervised on a labeled intent set, or a constrained grammar parser	intent accuracy	90%+ achievable with a bounded intent taxonomy

*Accuracy figures are indicative ranges based on published benchmarks for comparable datasets/tasks (e.g. METR-LA/PeMS-BAY for traffic, ASHRAE for energy) — actual figures must be established on your own held-out validation data and reported as such, not asserted in advance.

PART 6 — Dataset Strategy (Task 6)

Category	Datasets	Notes
Traffic	METR-LA, PeMS-BAY (benchmarking), SUMO-simulated Hyderabad data	synthetic data must eventually be cross-checked against any available real probe/GPS data
Accidents	State/city open accident registries where published; otherwise simulate accident events within SUMO scenarios	label scarcity is the main risk here — flag clearly if using simulated labels
Road network	OpenStreetMap (Geofabrik Telangana extract)	one-time <code>.osm.pbf</code> download, converted via <code>osmnx</code>
Weather	IMD station data, ERA5 reanalysis, Kaggle Indian-city weather sets	bulk download, not live
Pollution	CPCB CCR archive / Kaggle "Air Quality Data in India"	real gaps exist — document imputation choices explicitly
Energy	ASHRAE Great Energy Predictor III, UCI Individual Household Electric Power Consumption	building-level and household-level, respectively
GIS / land use	OSM building footprints, government land-use maps where published	
Population	Census/ward-level population data (Census of India, or municipal ward data)	
Satellite imagery	Landsat/Sentinel (thermal + optical bands) via Copernicus/USGS Earth Explorer	one-time bulk extract per region/season
Government open datasets	data.gov.in, state open-data portals	check licensing before redistribution
IoT sensor data	simulated/proxy until real sensors are deployed; smart-meter data from UCI as a stand-in for street-light/building energy	
Historical	multi-year spans wherever available, to support chronological splits	
Crowdsourced	citizen complaint logs (once the platform is live), can bootstrap from public grievance-portal data if available	

Preprocessing pipeline (applies across domains):

1. Resample to a consistent time bin per domain (5-15 min traffic, hourly energy/pollution).
2. Normalize per-channel (z-score), fit only on the training split.
3. Merge cross-domain features on station/ward/time keys (e.g. traffic volume into the pollution training frame).
4. Explicitly document missing-data handling (impute vs. drop) per dataset — CPCB gaps especially.
5. Feature engineering: calendar features (hour, day-of-week, holiday, festival flag), spatial features (adjacency/graph structure for GNNs).
6. Labeling: rule-based or historically-derived labels for anomaly/incident tasks where no ground truth exists (documented as a limitation).

7. **Chronological train/val/test split for every time-dependent task — never random shuffle.**
8. **Augmentation:** for CV tasks, standard image augmentation (crop/flip/brightness); for time series, avoid synthetic augmentation that could leak unrealistic patterns — prefer more real historical range.
9. **Storage:** raw bulk downloads in object storage (MinIO), processed/feature tables in PostgreSQL or TimescaleDB, embeddings in ChromaDB.

PART 7 — Technology Stack (Task 7)

Layer	Recommendation
Frontend	React + Leaflet (existing choice is solid)
Backend	FastAPI (Python) across all services — consistent with the existing model-serving pattern
AI/ML	PyTorch (traffic/pollution/weather/CV), XGBoost/LightGBM (energy), scikit-learn (classical baselines)
GIS	PostGIS, GeoPandas, osmnx, Leaflet vector tiles
Authentication	Keycloak (self-hosted OAuth2/OIDC + RBAC) — avoids depending on an external identity provider, consistent with the platform's "no external dependency" philosophy
Cloud	any Kubernetes-capable environment (on-prem or a sovereign/government cloud where required for data residency)
Storage	MinIO (S3-compatible object storage) for raw datasets, imagery, model artifacts
Caching	Redis (query cache, precomputed raster/heatmap cache)
Streaming/Queue	Kafka or, for a smaller academic-scale deployment, RabbitMQ — carries anomaly alerts, retraining triggers, GIS layer updates
Visualization	Leaflet (maps), Recharts/D3 (dashboards)
Notifications	self-hosted SMTP + a WebSocket channel for in-app alerts (avoids a paid third-party notification API)
Deployment	Docker Compose for dev, Kubernetes for production scale
CI/CD	GitHub Actions (build, test, train-pipeline validation, container publish)
Monitoring	Prometheus + Grafana (service metrics), OpenTelemetry (tracing), Loki or ELK (logs)
Containerization	Docker
Message queue	Kafka/RabbitMQ (see Streaming above)
Model serving	FastAPI for simplicity now; BentoML or Triton Inference Server as a next step if request volume grows enough to need batching/GPU sharing

PART 8 — Database Architecture (Task 8)

Store	Use	Reasoning
PostgreSQL + PostGIS	static/vector GIS layers (roads, hospitals, schools, wards, zoning), user/RBAC data, policy history	relational integrity + native geospatial queries
TimescaleDB	time-series sensor/model-input data (traffic speed, AQI readings, energy load)	Postgres alone degrades on high-frequency time-series at city scale; Timescale is a Postgres extension, so it stays compatible with PostGIS in the same ecosystem
Redis	precomputed raster/heatmap cache, session cache, hot query cache	sub-millisecond reads for dashboard tiles and session tokens
Elasticsearch	full-text search over complaints, policy documents, RAG-adjacent document search	needed once complaint volume/document corpus grows beyond simple SQL LIKE queries
Neo4j	<i>optional</i> — only justified if road-network/drainage-graph queries become genuinely graph-native (multi-hop routing, dependency chains); PostGIS + a GNN library can cover most needs without adding a fourth database, so treat this as a later addition, not a day-one requirement	
MinIO / object storage	raw dataset downloads, satellite imagery, model artifacts (<code>(.pt)</code> / <code>(.pkl)</code>), camera frame archives	large binary objects don't belong in a relational or time-series store
ChromaDB	RAG document embeddings (policy references, regulations, prior decisions)	already in the existing architecture, keep as-is

Data-type → store mapping: GIS → PostGIS · Time series → TimescaleDB · User/auth data → PostgreSQL · AI predictions/policy history → PostgreSQL (structured, queryable, auditable) · Historical bulk data → object storage (MinIO), summarized into TimescaleDB/PostGIS as needed · Logs → Loki/ELK, not the primary database · Images/camera frames → MinIO · Model artifacts → MinIO with a registry table in PostgreSQL tracking versions/metrics.

PART 9 — Authentication & Authorization (Task 9)

- **Identity provider:** self-hosted Keycloak (OAuth2/OIDC), issuing JWTs consumed by every backend service.
- **RBAC:** roles mapped to the platform's actual user types — Urban Planner, Municipal Official, Smart City Admin, Traffic Control Operator, Pollution Board Officer, Energy Authority Officer, Emergency Response Coordinator — each scoped to the modules/wards they're permitted to act on.
- **Departmental permissions:** row-level scoping (e.g. Postgres row-level security keyed on department/ward) so a Traffic Control operator cannot approve an Energy policy.
- **MFA:** TOTP-based (via Keycloak's built-in MFA) required for any Admin or policy-approval role.
- **Session management:** short-lived access tokens + refresh tokens, revocation list checked at the API gateway.

- **Password policy:** minimum length/complexity enforced at the identity provider, rotation policy for privileged accounts.
 - **Audit logs:** every policy approval, override, and data-export action logged with actor, timestamp, and before/after state, stored in an append-only PostgreSQL table (or a dedicated audit store) — separate from the model version log already in the original design.
 - **Admin portal:** a dedicated internal screen for user/role management, audit log review, and department-permission configuration — not exposed on the public-facing planner dashboard.
-

PART 10 — AI Agent Architecture (Task 10)

Agent roles: Orchestrator (planner/dispatcher), Traffic / Pollution / Energy / Weather (domain forecasters), Anomaly Monitors (one per domain), Arbitration (multi-objective optimizer), Policy Synthesis (structured output writer), Verification (feasibility/compliance checker).

Planning: replace the LLM-based planner with a **bounded intent classifier** over a fixed set of request types (e.g. "optimize traffic+pollution for ward X," "generate weekly domain report," "simulate disaster scenario Y") plus a deterministic DAG template per intent type — this is what actually removes the external-API dependency flagged in Section 1.2.

Memory: short-term (current request context, held in the Orchestrator process) and long-term (policy history + outcomes in PostgreSQL, retrievable by the Verification/Policy Synthesis agents for precedent-based reasoning).

Knowledge base: ChromaDB holds embedded policy documents, regulations, and past decisions; retrieval is scoped by department/ward to keep it relevant and auditable.

Decision engine: Arbitration Agent (NSGA-II) unchanged from the existing design — this part of the original architecture is genuinely solid and doesn't need replacing.

Recommendation engine: Policy Synthesis Agent, now templated (not LLM-generated free text) so every policy card's "reasoning" section is built from the same feature-contribution explanation format described in Section 3.3 — traceable and defensible in front of a government reviewer.

Inter-agent communication: internal REST for direct request/response (Orchestrator → domain agents), plus the new event bus (Kafka/RabbitMQ) for asynchronous signals — anomaly alerts, retraining triggers, GIS layer refresh notices — decoupling the monitoring/alerting path from the synchronous policy-generation path.

Model serving: FastAPI per model today; migrate to BentoML/Triton once request volume justifies batched/GPU-shared serving.

Retraining pipeline: scheduled batch retraining (as in the original design) plus an **automatic drift trigger** — when a domain's live validation error on the newest data batch exceeds a set threshold versus the currently deployed model, a retrain job is queued automatically rather than waiting for the fixed weekly schedule.

PART 11 — Development Roadmap (Task 11)

Phase	Objectives	Key Deliverables	Tools/Frameworks	Timeline*
1. Requirement analysis	finalize scope, user roles, success metrics	requirements doc, user workflows	—	1–2 weeks
2. Dataset collection	bulk-download all domain + contextual datasets	raw datasets in object storage	MinIO, requests/wget scripts	2–3 weeks
3. Data cleaning & feature engineering	resample, impute, merge cross-domain features	processed feature tables	pandas, GeoPandas	2–3 weeks
4. Database design	stand up PostGIS/TimescaleDB/Redis/MinIO schemas	schema migrations	PostgreSQL, TimescaleDB	1–2 weeks
5. GIS integration	road graph, building footprints, static layers	GIS API gateway, base layers	osmnx, PostGIS	2 weeks
6. Core model training	train Traffic/Pollution/Energy/Weather models	validated model artifacts	PyTorch, XGBoost	4–6 weeks
7. Model serving & orchestrator rebuild	FastAPI services, replace LLM orchestrator with trained planner	model-service containers, planner classifier	FastAPI, Docker	3–4 weeks
8. Arbitration & policy synthesis	NSGA-II integration, templated policy cards	working recommendation pipeline	pymoo	2–3 weeks
9. Authentication & security	Keycloak, RBAC, audit logging	secured API layer	Keycloak	2 weeks
10. Frontend/dashboard	planner dashboard, GIS layers, alerts	React + Leaflet app	React, Leaflet	4–5 weeks
11. Testing	unit/integration tests, model validation reports	test suite, validation report	pytest	2 weeks (ongoing)
12. Deployment	containerized deployment, CI/CD	deployed staging environment	Docker, GitHub Actions	1–2 weeks
13. Monitoring & observability	metrics/logs/tracing live	Grafana dashboards	Prometheus, Grafana	1–2 weeks
14. Retraining automation	scheduled + drift-triggered retraining	automated retrain pipeline	Airflow/cron + drift check	2 weeks
15. Future scaling	multi-ward/multi-city partitioning plan	scaling design doc	—	ongoing

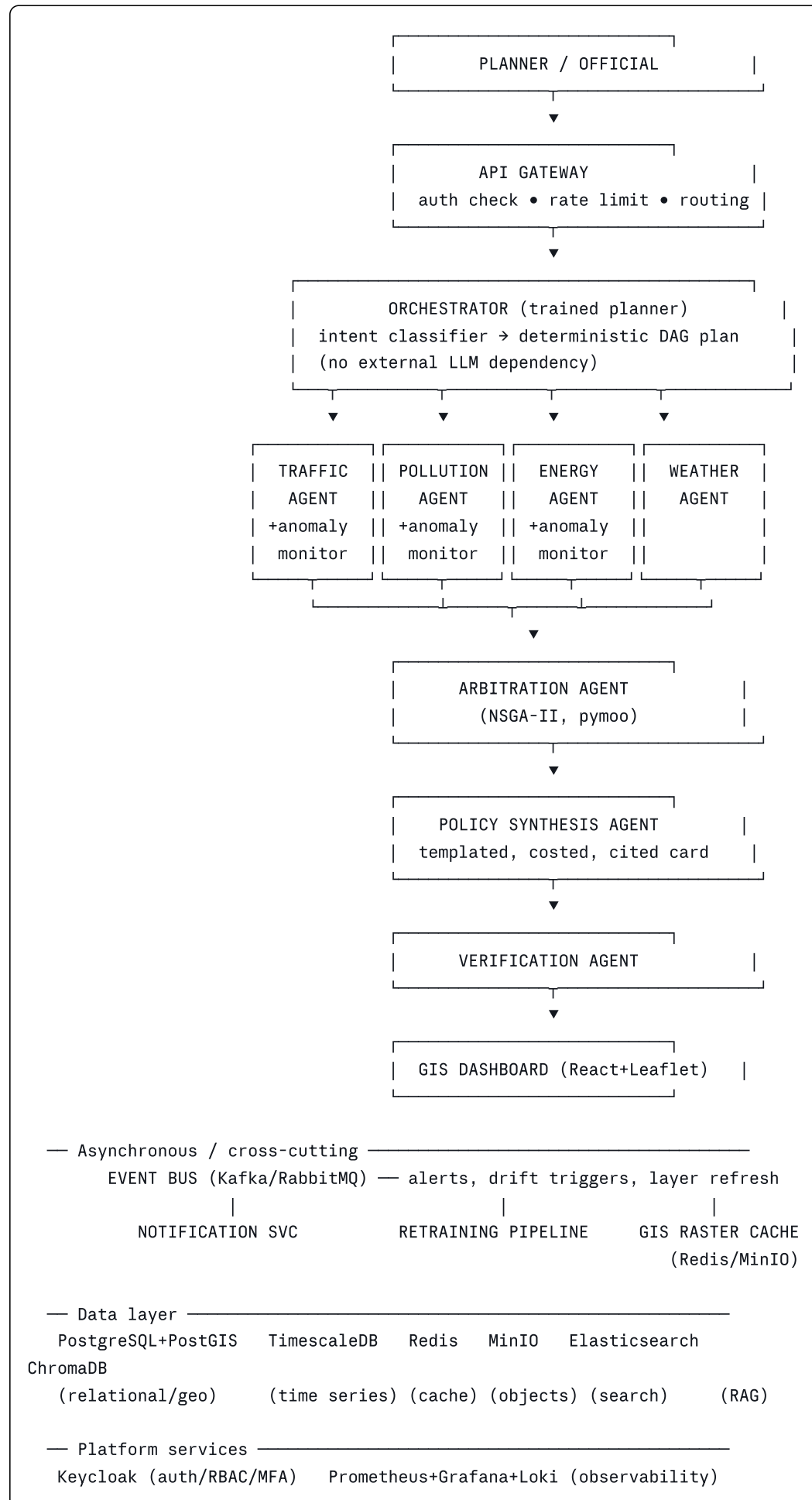
*Timeline assumes a single dedicated developer working part-time alongside coursework; compress by 30–40% with a small team, or treat these as effort estimates to plan sprints around rather than a fixed calendar.

PART 12 — Architecture Improvements (Task 12)

12.1 What changes and why

1. **Remove the LLM-dependent Orchestrator**, replacing it with a trained intent classifier + deterministic planner (Section 10) — resolves the core contradiction found in Section 1.2 and keeps the entire platform on locally-trained models end to end.
2. **Add an event bus** (Kafka/RabbitMQ) so anomaly alerts, retraining triggers, and GIS layer refreshes are asynchronous rather than stacked onto the synchronous policy-generation path.
3. **Split the database layer** into PostgreSQL+PostGIS (relational/geo), TimescaleDB (time series), Redis (cache), MinIO (objects), Elasticsearch (search) — instead of one Postgres instance carrying everything.
4. **Add Auth/RBAC/audit** as first-class services (Keycloak + row-level security + audit log table) rather than an afterthought.
5. **Add a Weather Agent and per-domain Anomaly Monitors** as first-class agents, not bolted onto the orchestrator.
6. **Precompute and cache GIS raster layers** (dispersion surfaces, heatmaps) instead of recomputing per request.
7. **Add an API gateway** in front of all services for rate limiting, auth enforcement, and a single ingress point.
8. **Add observability** (Prometheus/Grafana/Loki) so the platform's own health is monitored, not just the city's.

12.2 Improved architecture (text diagram)



12.3 What to keep unchanged

The domain-model choices (GNN/LSTM traffic, LSTM/TFT+Gaussian-plume pollution, XGBoost energy), the offline-training-then-local-serving pattern, the cross-domain feature

coupling (Traffic→Pollution, Energy→Pollution), and the Arbitration→Policy
Synthesis→Verification pipeline are all sound and don't need re-architecting — the changes
above are additive/corrective, not a rewrite.